

WordCount.java

```
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.fs.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.*;
import org.apache.hadoop.mapreduce.lib.output.*;
import org.apache.hadoop.util.*;

public class WordCount extends Configured implements Tool
{
    public static void main(String args[]) throws Exception {
        int res = ToolRunner.run(new WordCount(), args);
        System.exit(res);
    }

    public int run(String[] args) throws Exception {
        Path inputPath = new Path(args[0]);
        Path outputPath = new Path(args[1]);

        Configuration conf = getConf();
        Job job = new Job(conf, this.getClass().toString());
        job.setJarByClass(WordCount.class);

        FileInputFormat.setInputPaths(job, inputPath);
        FileOutputFormat.setOutputPath(job, outputPath);

        job.setJobName("WordCount");

        job.setMapperClass(Map.class);
        job.setCombinerClass(Reduce.class);
        job.setReducerClass(Reduce.class);
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);
        return job.waitForCompletion(true) ? 0 : 1;
    }
}
```

```

    public static class Map extends Mapper<LongWritable, Text, Text,
IntWritable>
    {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();
        @Override
        public void map(LongWritable key, Text value, Mapper.Context
context) throws IOException, InterruptedException
        {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                context.write(word, one);
            }
        }
    }
}

```

```

    public static class Reduce extends Reducer<Text, IntWritable, Text,
IntWritable>
    {
        @Override
        public void reduce(Text key, Iterable<IntWritable> values,
Context context) throws IOException, InterruptedException
        {
            int sum = 0;
            for(IntWritable value : values) sum += value.get();
            context.write(key, new IntWritable(sum));
        }
    }
}

```

LogFileAnalysis.scala

```
import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
object LogFileAnalysis {
  def main(args: Array[String]) {
    val conf = new SparkConf()
      .setAppName("Log Analysis")
      .setMaster("local[*]")
    val sc = new SparkContext(conf)
    val logData = sc.textFile("log.txt")
    val logLevels = logData.map(line => {
      if (line.contains("ERROR")) "ERROR"
      else if (line.contains("WARN")) "WARN"
      else if (line.contains("INFO")) "INFO"
      else "OTHER"
    })
    val counts = logLevels
      .map(level => (level, 1))
      .reduceByKey(_ + _)
    counts.collect().foreach(println)
    sc.stop()
  }
}
```

Log.txt

```
INFO Application started
ERROR Database failed
WARN Low memory
INFO User login
```

```
└─(vmsin0@martian)-[~/STUDY/Manual/DSBDA/EXP12]
└─$ scalac LogFileAnalysis.scala && scala LogFileAnalysis
INFO 2
ERROR 1
WARN 1
OTHER 0
```


WordCountExample.scala

```
import org.apache.spark.sql.SparkSession
object WordCountExample {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder()
      .appName("Word Count Example")
      .master("local[*]")
      .getOrCreate()
    val input = spark.read.textFile("input.txt")
    val wordCount = input
      .flatMap(line => line.split(" "))
      .groupByKey(word => word)
      .count()
    wordCount.show()
    spark.stop()
  }
}
```

Input

Hello Spark
Hello Scala
Spark is fast

Output

```
+-----+-----+
|value|count|
+-----+-----+
|Hello|  2  |
|Spark|  2  |
|Scala|  1  |
|is   |  1  |
|fast |  1  |
+-----+-----+
```